

C++ 연관 컨테이너 완전 학습 가이드

std::map과 std::unordered_map의 개념, 주요 함수, 시간 복잡도와 선택 기준을 한 권에 정리

정렬과 범위 검색이 필요한가? 아니면 키 기반 조회 속도가 중요한가?
이 질문에 답할 수 있다면 두 컨테이너의 선택은 거의 끝난다.

```
#include <map> #include <unordered_map>
```

이 가이드에서 배우는 것

두 컨테이너는 모두 고유한 키와 값을 연결하지만, 내부 구조와 제공 기능이 다릅니다. 단순히 “평균 $O(1)$ 이 더 빠르다”로 끝내지 않고, 실제 코드에서 안전하게 선택하고 사용하는 기준을 익힙니다.

장	주제	학습 목표
01	핵심 개념	트리와 해시 테이블의 차이 이해
02	생성 · 삽입 · 접근	[]의 부작용과 안전한 접근법 구분
03	검색 · 삭제 · 순회	iterator 기반 코드를 정확히 작성
04	정렬 · 범위 · 버킷	컨테이너별 고유 기능 활용
05	복잡도 · 선택 가이드	문제 조건에 맞는 컨테이너 선택
06	실전 패턴 · 체크리스트	빈도 계산과 흔한 실수 예방

30초 요약

std::map은 키를 정렬된 상태로 유지하는 균형 탐색 트리 기반 컨테이너입니다. 탐색·삽입·삭제가 $O(\log N)$ 이고, 정렬 순회와 범위 검색이 강점입니다.

std::unordered_map은 해시 테이블 기반 컨테이너입니다. 탐색·삽입·삭제가 평균 $O(1)$ 이지만 최악 $O(N)$ 이며, 순회 순서는 의미가 없습니다. 버킷과 load factor 관리 기능을 제공합니다.

표준 관련 메모

이 문서는 C++20을 기준으로 contains()를 포함합니다. C++17 이하에서는 find() != end() 또는 count()를 사용하세요.

01. 핵심 개념

공통점

두 컨테이너 모두 키-값 쌍을 저장하는 연관 컨테이너입니다. 키는 고유하며 원소의 자료형은 `std::pair<const Key, T>`입니다. 키가 `const`이므로 저장된 원소의 키를 직접 수정할 수 없습니다.

```
std::map<std::string, int> ordered;
std::unordered_map<std::string, int> hashed;

ordered["Kim"] = 100;
hashed["Kim"] = 100;
```

std::map: 정렬된 트리

일반적으로 균형 이진 탐색 트리로 구현됩니다. 표준은 특정 트리 종류를 강제하지 않지만, 연산의 로그 시간과 정렬 순회를 보장합니다. 키 비교에는 기본적으로 `std::less<Key>`가 사용됩니다.

std::unordered_map: 해시 테이블

해시 함수가 키를 버킷으로 보냅니다. 같은 버킷에 여러 키가 들어가는 해시 충돌이 생길 수 있습니다. 키가 같은 지는 기본적으로 `std::equal_to<Key>`로 판단합니다.

항목	std::map	std::unordered_map
필요 헤더		
내부 모델	정렬된 균형 탐색 트리	해시 테이블
순회 순서	비교 기준에 따른 정렬 순서	미지정 순서
키 요구사항	엄격 약순서 비교 가능	해시 가능 + 동등 비교 가능
강점	범위 검색, 안정적 로그 성능	평균적으로 빠른 정확 키 조회

중요

`unordered_map`은 `rbegin()/rend()`을 제공하지 않습니다. 또한 `equal_range()`는 두 컨테이너 모두 제공하지만, 해시 컨테이너에서는 '같은 키의 범위'를 뜻할 뿐 정렬 범위 검색이 아닙니다.

02. 생성 · 삽입 · 접근

operator[]

map $O(\log N)$ ·
unordered 평균 $O(1)$

역할 키에 대응하는 값에 접근하거나 값을 갱신합니다.

핵심 키가 없으면 기본값으로 원소를 삽입합니다. 단순 존재 확인에 사용하면 컨테이너가 변할 수 있습니다.

```
score["Kim"] = 100;
score["Kim"] += 5;
int x = score["Park"]; // inserts {"Park", 0}
```

at()

map $O(\log N)$ ·
unordered 평균 $O(1)$

역할 존재하는 키의 값에 접근합니다.

핵심 키가 없으면 std::out_of_range 예외를 던지며, 새 원소를 만들지 않습니다. const 컨테이너에서도 사용 가능합니다.

```
try {
    std::cout << score.at("Kim");
} catch (const std::out_of_range&) {
    std::cout << "missing";
}
```

insert()

map $O(\log N)$ ·
unordered 평균 $O(1)$

역할 키-값 쌍을 삽입합니다.

핵심 중복 키이면 기존 값을 유지합니다. 반환값의 bool로 삽입 성공 여부를 확인할 수 있습니다.

```
auto [it, inserted] = score.insert({"Lee", 90});
if (!inserted) {
    std::cout << "already exists: " << it->second;
}
```

emplace()

map $O(\log N)$ ·
unordered 평균 $O(1)$

역할 전달한 인수로 원소를 컨테이너 안에서 생성합니다.

핵심 불필요한 임시 객체를 줄일 수 있지만 중복 키 검사 과정에서 객체 생성 비용이 생길 수 있습니다.

```
auto [it, ok] = score.emplace("Park", 80);
```

02. 생성 · 삽입 · 접근 (계속)

try_emplace() · C++17

map $O(\log N)$ ·
unordered 평균 $O(1)$

역할 키가 없을 때만 mapped value를 생성해 삽입합니다.

핵심 중복 키이면 값 생성 인수를 사용하지 않습니다. 생성 비용이 큰 값에 유리합니다.

```
auto [it, ok] = cache.try_emplace(key, expensive_value);
```

insert_or_assign() · C++17

map $O(\log N)$ ·
unordered 평균 $O(1)$

역할 키가 없으면 삽입하고, 있으면 값을 대입합니다.

핵심 operator[]과 달리 값이 기본 생성 가능할 필요가 없습니다. 반환값 bool은 새 삽입 여부입니다.

```
auto [it, inserted] = score.insert_or_assign("Kim", 95);
```

어떤 삽입 함수를 선택할까?

의도	권장 표현	이유
없으면 기본값 생성 후 수정	<code>m[key]++</code>	빈도 계산에 간결
중복이면 기존 값 유지	<code>insert / emplace</code>	성공 여부 확인 가능
중복이면 값 덮어쓰기	<code>insert_or_assign</code>	의도가 명확
값 생성 비용이 큼	<code>try_emplace</code>	중복 시 값 생성 회피
반드시 존재해야 함	<code>at</code>	실수로 삽입하지 않음

초기화 예제

```
std::map<std::string, int> score {
    {"Kim", 100},
    {"Lee", 90},
    {"Park", 80}
};

std::unordered_map<int, std::string> name {
    {1, "Kim"}, {2, "Lee"}
};
```

03. 검색 · 존재 확인

find()

map $O(\log N)$ ·
unordered 평균 $O(1)$, 최악 $O(N)$

역할 키를 찾아 iterator를 반환합니다.

핵심 없으면 end()를 반환합니다. 존재 확인 후 값까지 사용할 때 가장 적합합니다.

```
auto it = score.find("Kim");
if (it != score.end()) {
    std::cout << it->first << ": " << it->second;
}
```

contains() · C++20

map $O(\log N)$ ·
unordered 평균 $O(1)$, 최악 $O(N)$

역할 키의 존재 여부를 bool로 반환합니다.

핵심 iterator나 값이 필요 없고 존재 여부만 읽고 싶을 때 가장 명확합니다.

```
if (score.contains("Kim")) {
    std::cout << "exists";
}
```

count()

map $O(\log N)$ ·
unordered 평균 $O(1)$, 최악 $O(N)$

역할 주어진 키를 가진 원소 수를 반환합니다.

핵심 map과 unordered_map은 키가 고유하므로 결과는 0 또는 1입니다. multi 계열에서는 여러 개일 수 있습니다.

```
if (score.count("Kim") != 0) {
    std::cout << "exists";
}
```

검색 표현 선택

값까지 사용: find() · 존재 여부만: contains() · C++17 이하: find() 또는 count(). 존재 확인 뒤 다시 at()을 호출 하면 검색이 두 번 일어나므로 iterator를 재사용하는 편이 좋습니다.

03. 삭제 · 크기 · 순회

erase()

키: map $O(\log N)$,
unordered 평균 $O(1)$

역할 키, iterator 또는 iterator 범위의 원소를 삭제합니다.

핵심 키 버전은 삭제된 원소 수를 반환합니다. iterator 버전은 삭제 다음 원소를 가리키는 iterator를 반환합니다.

```
score.erase("Lee");
for (auto it = score.begin(); it != score.end(); ) {
    if (it->second < 60) it = score.erase(it);
    else ++it;
}
```

clear() · size() · empty()

clear $O(N)$ · size/empty
 $O(1)$

역할 전체 삭제, 원소 수 확인, 비어 있는지 확인을 담당합니다.

핵심 clear는 모든 원소를 파괴합니다. size와 empty는 상수 시간입니다.

```
std::cout << score.size();
if (!score.empty()) score.clear();
```

순회

```
for (const auto& [key, value] : score) {
    std::cout << key << ": " << value << '\n';
}

for (auto& [key, value] : score) {
    value += 5; // value can change
    // key cannot change
}
```

map은 키 정렬 순서로 순회합니다. unordered_map의 순회 순서는 미지정이며, 삽입·삭제·rehash 뒤 달라질 수 있습니다. 출력 순서나 테스트 결과를 여기에 의존하지 마세요.

iterator 무효화

연산	map	unordered_map
삽입	기존 iterator 유효	rehash가 일어나면 모든 iterator 무효
삭제	삭제한 원소만 무효	삭제한 원소만 무효
rehash / reserve	해당 없음	iterator 무효, 원소 참조·포인터는 유효

04. map 전용: 정렬과 범위 검색

<code>lower_bound(key)</code>	$O(\log N)$
역할 key 이상인 첫 원소의 iterator를 반환합니다.	
핵심 그런 원소가 없으면 <code>end()</code> 입니다. <code>[low, high]</code> 범위의 시작점을 찾는 데 유용합니다.	
<pre>auto it = m.lower_bound(15); // keys: 10, 20, 30 if (it != m.end()) std::cout << it->first; // 20</pre>	
<code>upper_bound(key)</code>	$O(\log N)$
역할 key보다 큰 첫 원소의 iterator를 반환합니다.	
핵심 상한이 포함되지 않는 범위를 만들거나 특정 키 다음 위치를 찾을 때 사용합니다.	
<pre>auto it = m.upper_bound(20); if (it != m.end()) std::cout << it->first; // 30</pre>	
<code>equal_range(key)</code>	map $O(\log N)$ · unordered 평균 $O(1)$
역할 <code>lower_bound</code> 와 <code>upper_bound</code> 에 해당하는 iterator 쌍을 반환합니다.	
핵심 map에서는 고유 키라 범위 길이가 0 또는 1입니다. unordered_map에도 있으나 정렬 범위 의미는 아닙니다.	
<pre>auto [first, last] = m.equal_range(20); for (auto it = first; it != last; ++it) { std::cout << it->second; }</pre>	

구간 [15, 30] 출력

```
for (auto it = m.lower_bound(15);
     it != m.upper_bound(30);
     ++it) {
    std::cout << it->first << '\n';
}
```

내림차순 map

```
std::map<int, std::string, std::greater<int>> rank;
rank[3] = "C";
rank[1] = "A";
rank[2] = "B"; // iteration: 3, 2, 1
```


04. unordered_map 전용: 해시와 버킷

성능은 원소가 버킷에 얼마나 고르게 분산되는지에 영향을 받습니다. load factor = 원소 수 / 버킷 수이며, 임계값을 넘으면 구현체가 버킷을 늘리는 rehash를 수행할 수 있습니다.

함수	설명	활용
bucket_count()	현재 버킷 수	구조 관찰·진단
load_factor()	현재 load factor	밀집도 확인
max_load_factor()	rehash 기준이 되는 최대값	공간·속도 균형 조절
reserve(n)	n개 원소를 위한 버킷 확보	대량 삽입 전 rehash 감소
rehash(n)	버킷 수를 최소 n 이상으로 재구성	버킷 수 직접 제어
hash_function()	해시 함수 객체 반환	고급 진단·사용자 정의

대량 삽입 전 reserve

```
std::unordered_map<std::string, int> freq;
freq.reserve(100000);

for (const auto& word : words) {
    ++freq[word];
}
```

사용자 정의 키

```
struct Point { int x, y; };
bool operator==(const Point& a, const Point& b) {
    return a.x == b.x && a.y == b.y;
}
struct PointHash {
    std::size_t operator()(const Point& p) const noexcept {
        return std::hash<int>{}(p.x) ^
            (std::hash<int>{}(p.y) << 1);
    }
};
std::unordered_map<Point, int, PointHash> visits;
```

같다고 판단되는 두 키는 반드시 같은 해시 값을 만들어야 합니다. 반대 방향(같은 해시값이면 같은 키)은 필요하지 않으며, 충돌은 정상적으로 처리됩니다.

05. 시간 복잡도 비교

연산	std::map	std::unordered_map
find / contains / count	$O(\log N)$	평균 $O(1)$, 최악 $O(N)$
insert / emplace	$O(\log N)$	평균 $O(1)$, 최악 $O(N)$
operator[] / at	$O(\log N)$	평균 $O(1)$, 최악 $O(N)$
erase(key)	$O(\log N)$	평균 $O(1)$, 최악 $O(N)$
정렬 순회	자연스럽게 가능: $O(N)$	별도 정렬 필요: 보통 $O(N \log N)$
lower_bound / upper_bound	$O(\log N)$	제공하지 않음
전체 clear	$O(N)$	$O(N)$

복잡도 해석의 핵심

평균 $O(1)$ 은 항상 map보다 빠르다는 뜻이 아닙니다. 작은 데이터, 해시 계산 비용이 큰 키, 메모리 지역성, 충돌, rehash 비용에 따라 실제 성능은 달라집니다. 반면 map은 최악의 경우에도 로그 시간 보장이 필요할 때 예측하기 쉽습니다.

메모리 관점

map은 각 노드에 트리 연결 정보가 필요하고, unordered_map은 노드 외에 버킷 배열이 필요합니다. 어느 쪽이 항상 더 적은 메모리를 쓴다고 단정할 수 없으며 구현과 데이터 형태에 따라 다릅니다.

키 요건 비교

컨테이너	기본 요구	사용자 정의 타입
map	키 사이의 엄격 약순서	operator< 또는 comparator 제공
unordered_map	해시 + 동등성	std::hash 특수화 또는 hasher, == 제공

05. 선택 가이드

std::map을 선택하세요

- 키를 정렬된 순서로 순회해야 할 때
- 특정 키 이상/초과 원소나 키 범위를 찾아야 할 때
- 최악의 경우에도 $O(\log N)$ 성능 보장이 중요할 때
- 사용자 정의 순서(comparator)가 문제 의미와 직접 연결될 때

std::unordered_map을 선택하세요

- 정렬이 필요 없고 정확한 키 조회가 중심일 때
- 빈도 계산, 캐시, ID 기반 조회처럼 삽입·검색이 매우 잦을 때
- 키에 좋은 해시 함수가 있으며 평균 성능이 중요한 때
- 대량 데이터 수를 미리 알아 reserve로 rehash를 줄일 수 있을 때

결정 흐름

질문	예	아니오
정렬 순회나 범위 검색이 필요한가?	map	다음 질문
최악의 로그 성능 보장이 필요한가?	map	다음 질문
키를 안정적으로 해시할 수 있는가?	unordered_map 후보	map
프로파일링에서 조회가 병목인가?	unordered_map 비교 측정	가독성·요구사항 우선

대표 사용 사례

상황	추천	이유
단어 빈도 계산	unordered_map	정확 키 조회 중심
점수 구간 질의	map	lower/upper_bound 필요
학생 번호순 출력	map	정렬 순회
세션 캐시	unordered_map	평균 빠른 ID 조회
온라인 순위 경계 탐색	map	이웃·범위 탐색

06. 실전 패턴

패턴 1 · 빈도 계산

```
std::unordered_map<std::string, int> freq;
for (const auto& word : words) {
    ++freq[word];
}
if (auto it = freq.find("apple"); it != freq.end()) {
    std::cout << it->second;
}
```

패턴 2 · 정렬된 키 출력

```
std::map<int, std::string> students;
students.emplace(3, "Park");
students.emplace(1, "Kim");
students.emplace(2, "Lee");

for (const auto& [id, name] : students) {
    std::cout << id << ' ' << name << '\n';
} // 1, 2, 3
```

패턴 3 · 값 갱신 정책을 명확히

```
score.try_emplace("Kim", 100);           // keep old value
score.insert_or_assign("Kim", 95);       // overwrite old value
```

패턴 4 · 범위 내 원소 삭제

```
auto first = ordered.lower_bound(10);
auto last  = ordered.upper_bound(20);
ordered.erase(first, last); // erase keys in [10, 20]
```

패턴 5 · unordered_map 결과를 정렬 출력

```
std::vector<std::pair<std::string, int>> items(
    freq.begin(), freq.end());
std::sort(items.begin(), items.end());

for (const auto& [key, value] : items) {
    std::cout << key << ": " << value << '\n';
}
```

06. 흔한 실수와 체크리스트

실수	왜 문제인가	대안
존재 확인에 m[key] 사용	없는 키가 기본값으로 삽입됨	contains 또는 find
find 결과를 바로 역참조	없으면 end() 역참조로 미정의 동작	end와 비교 후 사용
unordered 순서에 의존	순서는 보장되지 않고 바뀔 수 있음	map 또는 별도 정렬
lower/upper_bound 결과 무검사	end()일 수 있음	end 확인
rehash 뒤 iterator 사용	모든 iterator가 무효화될 수 있음	다시 find하거나 reserve 선행
삭제 중 ++it 중복	원소 건너뛰기·무효 iterator 위험	it = erase(it)
unordered에 범위 질의 기대	정렬 관계가 없음	map 선택
평균 O(1)을 절대 성능으로 해석	충돌·키·메모리 영향 무시	실데이터로 측정

최종 체크

- ☐ 키 순서가 필요한가?
- ☐ 범위 검색이 필요한가?
- ☐ 없는 키를 조회할 때 삽입되어도 되는가?
- ☐ 중복 키일 때 유지할지 덮어쓸지 명확한가?
- ☐ iterator 무효화 조건을 고려했는가?
- ☐ unordered_map이라면 해시 품질과 reserve를 고려했는가?
- ☐ 실제 데이터 크기와 접근 패턴으로 성능을 측정했는가?

한 문장 결론

정렬·범위·예측 가능한 최악 성능은 std::map, 순서 없는 정확 키 조회의 평균 성능은 std::unordered_map. 함수 이름은 대부분 비슷하지만, 순서 보장과 iterator 무효화, 해시 관리 기능의 차이가 실제 코드 품질을 좌우합니다.

빠른 복습: []는 없으면 삽입 · at은 없으면 예외 · find는 iterator · contains는 bool · map은 lower/upper_bound · unordered_map은 reserve/load_factor.